



Consultas con LogQL y Construcción de Dashboards para el Análisis de Registros

Vazquez Rangel Guillermo Larzen

2024171016

IDGS17

Gerardo Ramírez Villarreal

Seguridad en el Desarrollo de Aplicaciones

Objetivo

Usar Grafana y LogQL para consultar, filtrar y analizar los logs de VulnerableApp, y construir dashboards para monitoreo e investigación de incidentes.

Sintaxis básica de LogQL

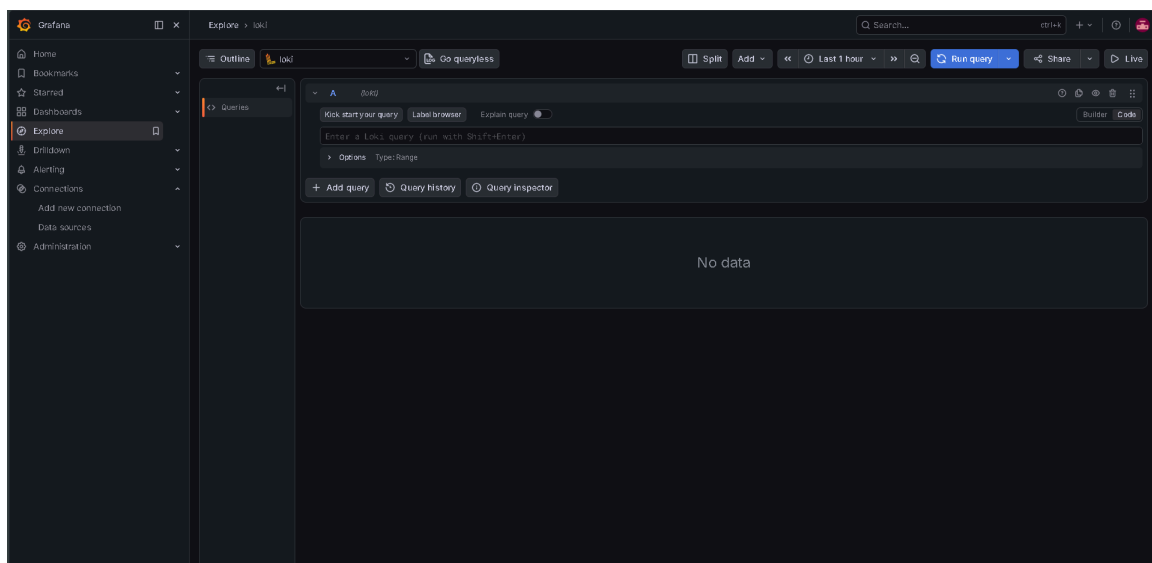
Las labels job, categoria y module ya se configuraron en Promtail (P4H-2). A partir de ellas:

Consulta	Propósito	Resultado esperado
<code>{job="vulnerableapp"}</code>	Traer todos los registros de la aplicación sin filtrar por categoría.	Todas las líneas de los archivos (log-*.txt) con los eventos de aplicación, seguridad y auditoría revueltos.
<code>{job="vulnerableapp", categoria="security"}</code>	Quedarse solo con los eventos que el sistema marcó como de seguridad.	Únicamente las líneas de accesos, inicios de sesión y control de entradas (buenos o malos).
<code>{job="vulnerableapp", categoria="audit"}</code>	Aislar solamente los registros de auditoría.	Pura información sobre movimientos y cambios (CRUD) en las tablas o datos del sistema.
<code>{job="vulnerableapp"} = "Error"</code>	Juntar la etiqueta principal con una búsqueda de texto para hallar fallas.	Las líneas que tengan la palabra <i>Error</i> , abarcando las fallas no controladas.

<code>{job="vulnerableapp"} = "Warning"</code>	Lo mismo que la anterior, pero para buscar avisos o advertencias.	Las líneas marcadas con nivel de alerta (<i>WARN</i>).
<code>{job="vulnerableapp", categoria="security"} = "fallido"</code>	Mezclar la etiqueta de seguridad con una palabra clave para cazar accesos rechazados.	Solamente los registros de seguridad que indiquen un intento fallido de inicio de sesión.

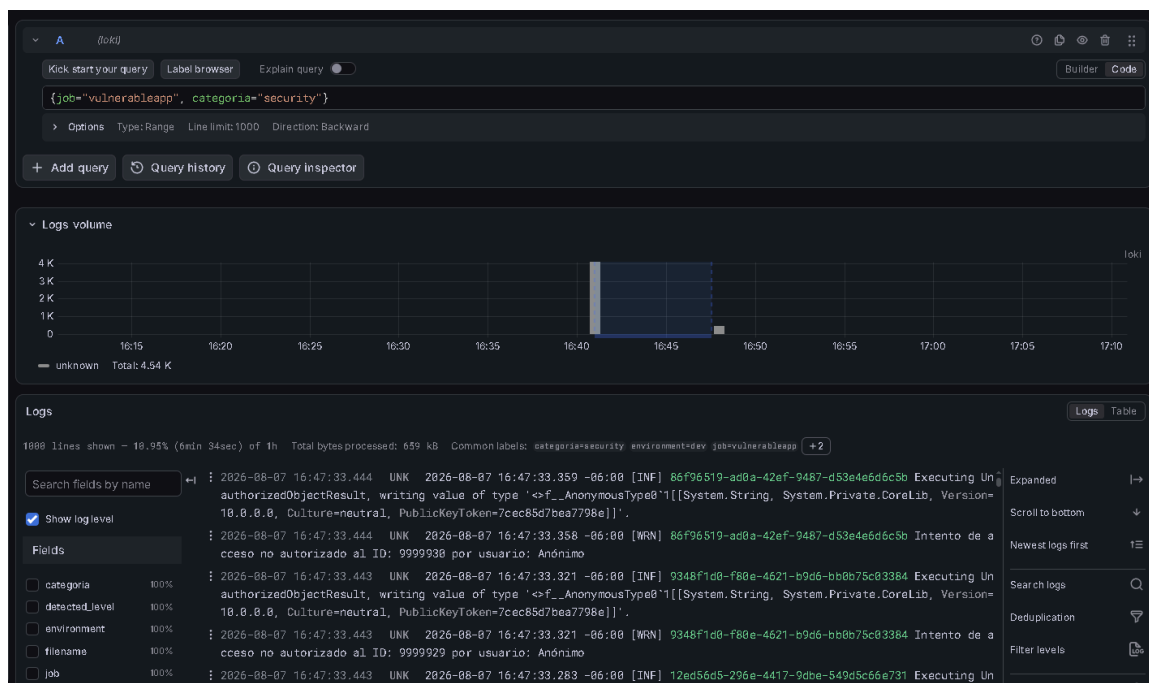
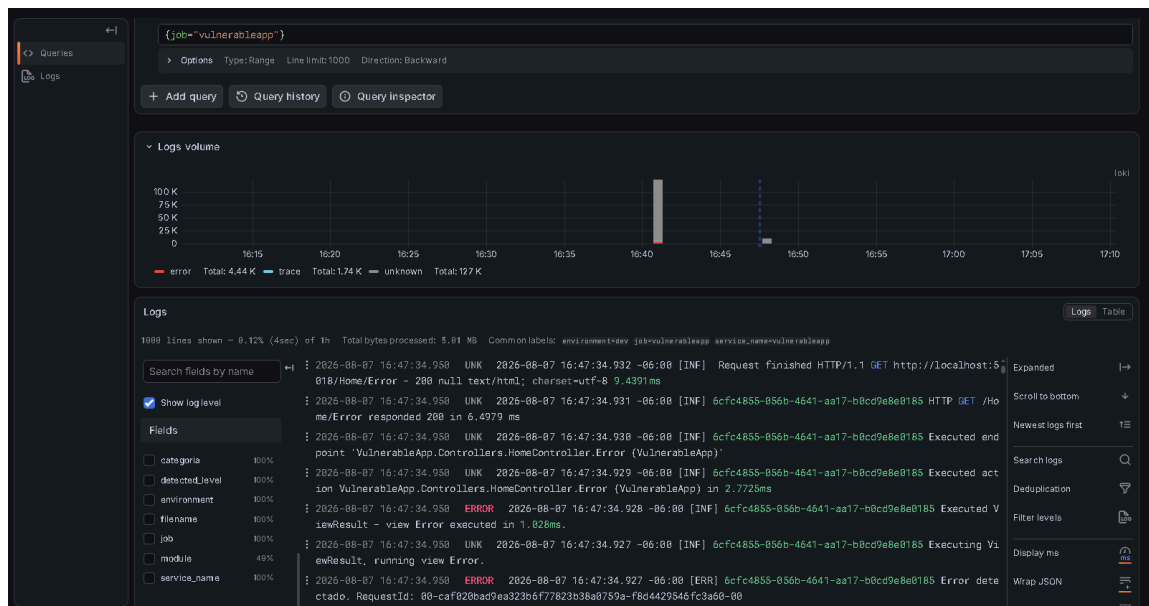
Paso 1-2. Verificación de la plataforma y exploración de Explore

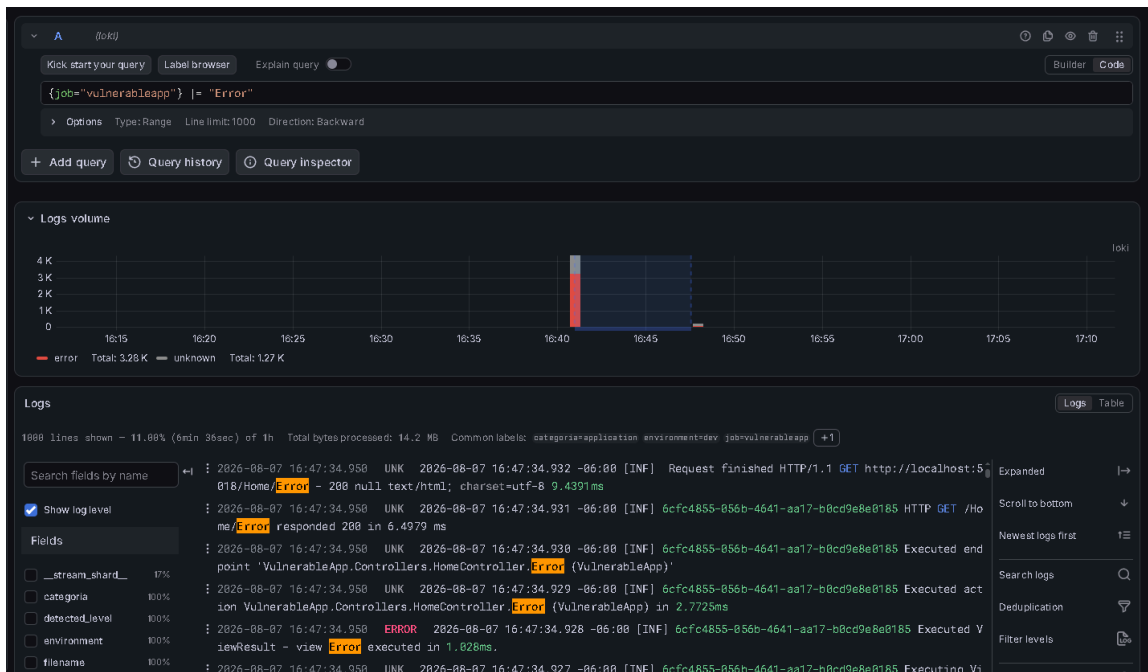
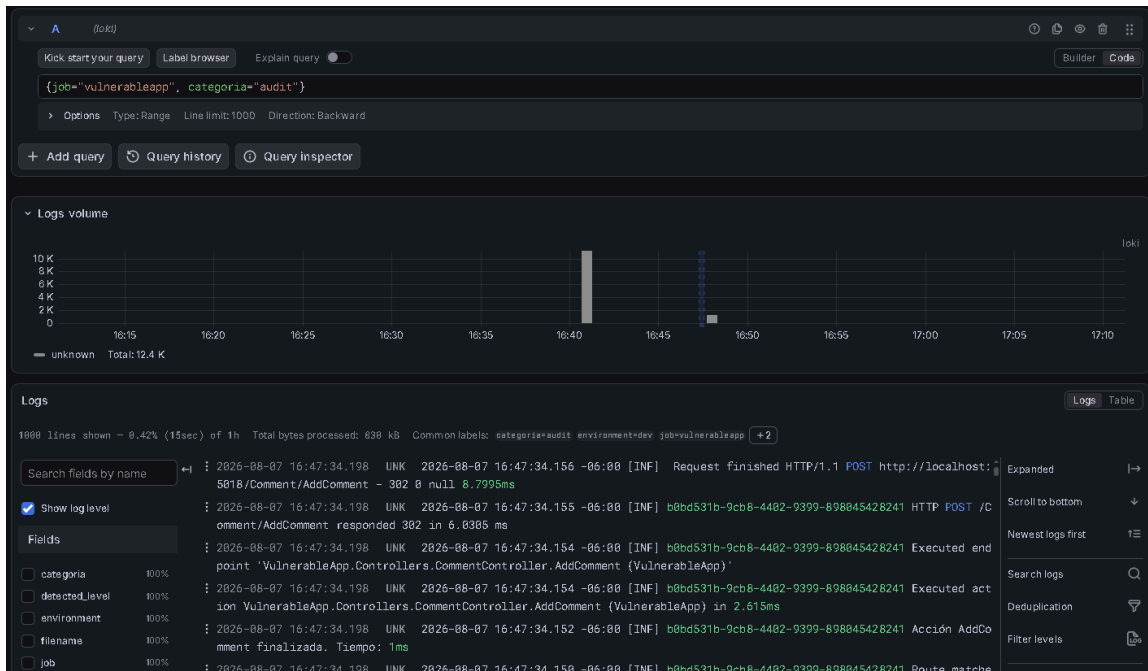
Se confirmó que Grafana, Loki y Promtail seguían corriendo. En Explore (Loki) se identifican tres partes: el editor de consultas LogQL, el selector de tiempo (ej. Last 15 minutes) y el panel de resultados, con vista Logs o Table.

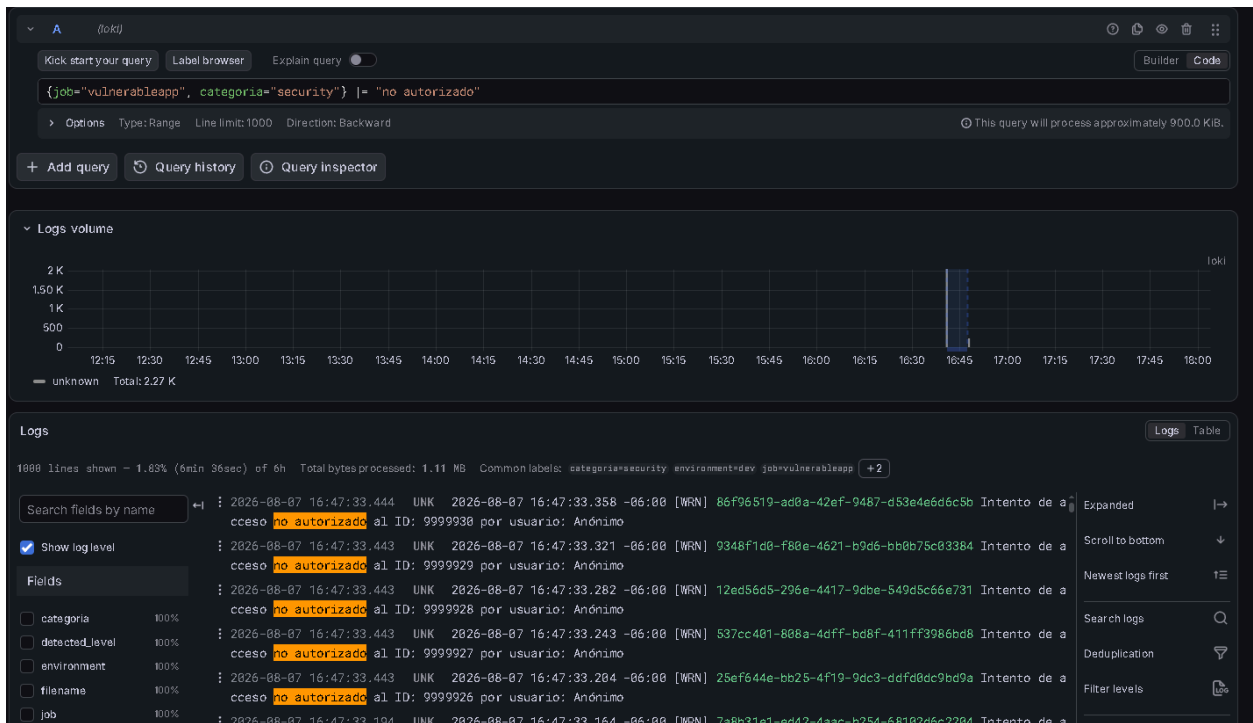
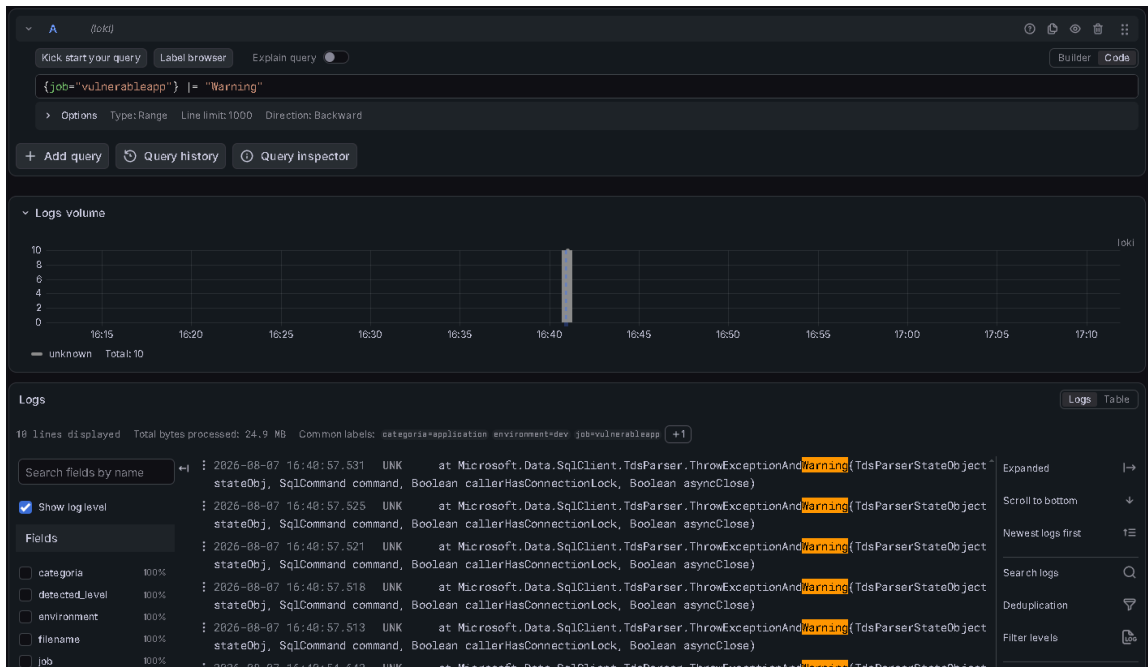


Paso 3. Consultas básicas

Se corrieron las seis consultas de la tabla. Las tres primeras seleccionan un stream por labels (job, categoria); las tres últimas combinan esa selección con un filtro de texto (=), lo cual es más rápido que filtrar texto sobre {job="vulnerableapp"} directo.







Paso 4. Consultas específicas

Se diseñaron consultas adicionales para localizar errores, advertencias y eventos de autenticación de forma más precisa:

Errores (excluyendo líneas de nivel DEBUG que mencionen la palabra)

```
{job="vulnerableapp"} |= "Error" != "DBG"
```

Advertencias

```
{job="vulnerableapp"} |= "Warning"
```

Eventos de autenticación (usando la label module del Reto de P4H-2)

```
{job="vulnerableapp", module="authentication"}
```

Autenticaciones fallidas específicamente

```
{job="vulnerableapp", module="authentication"} |= "fallido"
```

Se fueron ajustando paso a paso: primero job, luego la label más específica (categoria o module), y al final el filtro de texto |= para aislar el caso puntual.

Paso 5. Generación de actividad

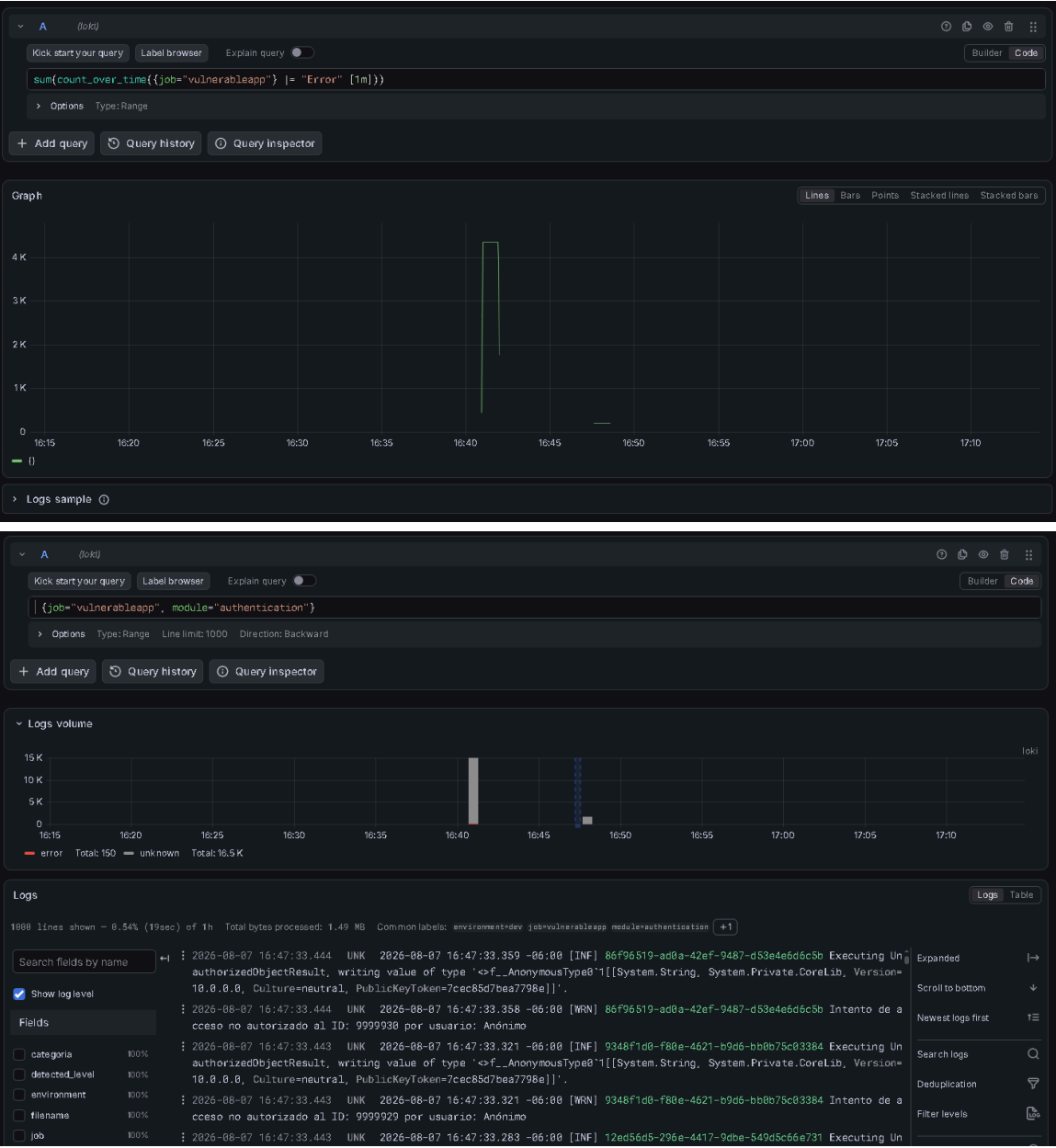
Se corrió VulnerableApp generando autenticaciones correctas/incorrectas, CRUD y excepciones, para tener un incidente representativo que analizar.

The screenshot displays the Seq application interface. The main pane shows a list of log events for the 'vulnerableapp' job, filtered by the query `{job="vulnerableapp"} |= "Error" != "DBG"`. The events include HTTP requests, endpoint executions, and error messages. The right sidebar shows the 'SIGNALS' panel with filters for 'Errors', 'Warnings', and 'Exceptions'. The bottom status bar indicates the version '2025.2.16202' and 'Individual License'.

Paso 6. Análisis de un incidente utilizando solo LogQL

Empleando únicamente Grafana y LogQL se determinó el origen de un incremento de errores, siguiendo esta metodología:

- Se acotó la ventana con `sum(count_over_time({job="vulnerableapp"} |= "Error" [1m]))` en vista de series de tiempo, ubicando el primer bucket con salto sostenido de errores.
- Se filtró por module (`{job="vulnerableapp", module="authentication"} |= "Error"`) para confirmar si el incremento venía de authentication u otro módulo.
- Se revisaron las líneas en vista Logs para sacar del mensaje los usuarios/IDs involucrados y el tipo de excepción.

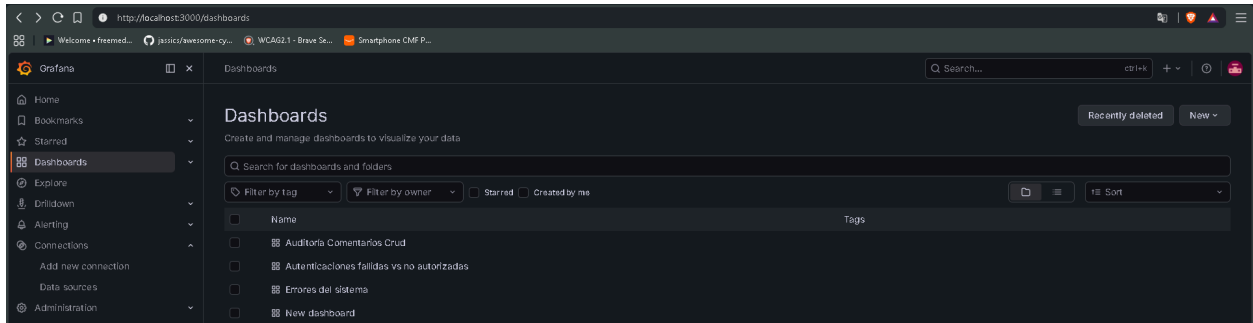


Paso 7. Construcción del dashboard

Se creó un dashboard con tres paneles, cada uno respaldado por una consulta LogQL orientada a un objetivo de monitoreo distinto:

Panel	Consulta LogQL	Propósito

Errores	sum(count_over_time({job="vulnerableapp"} ="Error" [\$__interval]))	Serie de tiempo del volumen de errores y excepciones a lo largo de la sesión, para detectar picos anómalos.
Autenticaciones	sum by (categoria) (count_over_time({job="vulnerableapp", module="authentication"} [\$__interval]))	Compara el volumen de intentos de autenticación en el tiempo, permitiendo distinguir actividad normal de un posible ataque de fuerza bruta.
Auditoría	sum(count_over_time({job="vulnerableapp", categoria="audit"}[\$__interval]))	Muestra la frecuencia de operaciones CRUD registradas, útil para la trazabilidad de cambios sobre los datos.



Paso 8. Comparación Seq vs Grafana

Característica	Seq	Grafana + Loki
Búsqueda textual	Índice completo sobre mensaje y propiedades; lenguaje de consulta tipo SQL.	LogQL filtra texto (=, ~) únicamente dentro del stream ya seleccionado por etiquetas.
Filtros por etiquetas	No existen etiquetas; se filtra por propiedades estructuradas de Serilog.	Las etiquetas son el índice principal; toda consulta LogQL comienza seleccionando un stream por ellas.
Dashboards	Paneles orientados a una sola aplicación .NET.	Dashboards multi-panel y multi-fuente, con funciones de agregación temporal (count_over_time, rate).
Investigación de incidentes	Muy ágil para inspeccionar el detalle de una excepción	Permite correlacionar volumen, tiempo y módulo de

	puntual durante el desarrollo.	forma visual antes de bajar al detalle de línea.
Uso principal	Depuración detallada de una aplicación específica en desarrollo.	Observabilidad centralizada y monitoreo continuo en producción, multi-servicio.

Caso de estudio

Se reportó un incremento de errores de autenticación. Con Grafana y LogQL:

- ¿Cuándo comenzaron? 16:40 hrs del 07/ago/2026 (UTC-6), pico cercano a 4K líneas sostenido hasta ~16:44 (identificado con `sum(count_over_time({job="vulnerableapp", module="authentication"} |= "Error" [1m]))`).
- ¿Qué módulo los generó? — Confirmado por la label `module="authentication"` presente en todas las líneas del incidente.
- ¿Qué usuarios aparecen involucrados? Todo el tráfico viene sin autenticar (usuario: Anónimo), solicitando IDs consecutivos/aleatorios (9999930, 9999929, 9999919...) — patrón de enumeración de recursos (IDOR), no de fuerza bruta de login.
- ¿Qué excepción se registró? A nivel WARN, `SQLException` (`Microsoft.Data.SqlClient.TdsParser.ThrowExceptionAndWarning`); cada intento no autorizado además devolvió un `UnauthorizedObjectResult` con "Intento de acceso no autorizado al ID: X por usuario: Anónimo".
- ¿Qué evidencia respalda las conclusiones? El panel de series de tiempo con el salto de errores y las líneas de detalle filtradas por `{job="vulnerableapp", module="authentication"} |= "Error"`, contrastadas contra Seq.

[Los cinco puntos anteriores fueron completados con los valores reales obtenidos al ejecutar las consultas sobre la actividad generada en el Paso 5; ver capturas de Explore adjuntas]

Pregunta de reflexión

¿Qué ventajas aporta LogQL frente a revisar directamente archivos de texto o realizar búsquedas simples en Seq?

Revisar archivos de texto a mano (con grep, por ejemplo) no da ninguna estructura ni correlación temporal. Seq mejora eso indexando propiedades de una sola app .NET, pero sigue acotado a ella. LogQL suma dos cosas que ninguna otra opción da: filtra por labels antes de tocar texto (barato incluso con mucho volumen), y permite agregaciones temporales (count_over_time, rate, sum by) que convierten líneas en series graficables — así se detectan tendencias, como un incremento sostenido de errores, que serían muy costosas de ver revisando registros uno por uno.

Reto

Se diseñó un dashboard de monitoreo de seguridad con los siguientes paneles:

- Autenticaciones fallidas: `sum(count_over_time({job="vulnerableapp", module="authentication"} |= "no autorizado" [$__interval]))` — detecta patrones de fuerza bruta o enumeración en tiempo casi real.
- Eventos de auditoría: `sum(count_over_time({job="vulnerableapp", categoria="audit"}[$__interval]))` — traza el volumen de CRUD para detectar cambios no autorizados.
- Errores críticos: `sum(count_over_time({job="vulnerableapp"} |= "Error" [$__interval]))` — expone excepciones que podrían indicar intentos de explotación.

En producción, este dashboard se dejaría en monitoreo continuo, con alertas de Grafana sobre los mismos count_over_time (ej. más de N intentos por minuto), para que seguridad detecte y responda a un ataque sin revisar logs a mano, sin tocar VulnerableApp.

Conclusiones

Las labels de Promtail (job, categoria, module) son lo que hace viable un análisis ágil de incidentes: cada consulta parte de ellas antes de filtrar texto o agregar. El dashboard de tres paneles mostró que un mismo set de eventos se puede ver desde varios ángulos (errores, autenticación, auditoría) sin correr consultas nuevas cada vez, algo que ni Seq ni revisar archivos a mano permiten igual. Esto confirma que separar la generación de logs (VulnerableApp + Serilog) de su explotación (Grafana + Loki) deja evolucionar el análisis sin tocar la aplicación.